10 — Decisões de projeto

> Código: todo o firmware · Relacionados: 02 — Arquitetura, MANUTENCAO.md

Registro das escolhas que dão forma ao código, com o motivo de cada uma. Antes de reverter uma delas, leia o motivo: quase todas resolvem um problema concreto. Ao mudar uma decisão, não apague o texto antigo: marque "Substituída por D-xx" e acrescente a nova no fim.

Cada decisão tem: **Contexto** (o problema), **Decisão**, **Alternativas** (o que ficou de fora e por quê), **Consequências** e **Código** (onde está).

D-01 — O colete funciona sozinho; o app é opcional

- **Contexto:** o uso real é o dia inteiro, longe do computador; o celular pode estar desligado ou sem rede.
- **Decisão:** toda a detecção e o alerta rodam no ESP32. O app só mostra dados e manda comandos. Calibração e ajustes também funcionam sem app (botão e serial), e os ajustes ficam salvos no colete.
- **Alternativas:** processar no celular ou na nuvem deixaria o colete inútil sem conexão.
- **Consequências:** o firmware precisa de retorno próprio ao usuário (padrões de vibração, 06).
- **Código:** `src/main.cpp`.

D-02 — App web embarcado: Access Point + HTTP + WebSocket

- **Contexto:** era preciso uma interface no celular sem instalar nada, sem internet e sem depender da rede da faculdade.
- **Decisão:** o ESP32 cria a própria rede Wi-Fi (Access Point `BioSpine`, IP 192.168.4.1), serve uma página HTML/CSS/JS de arquivo único (porta 80) e troca JSON por WebSocket (porta 81) a 10 Hz. A página vai embutida na flash (`board_build.embed_txtfiles`) e não usa CDN.
- **Alternativas:** app nativo (Android/iOS) e BLE: mais trabalho e instalação; nuvem/MQTT: precisa de internet. Ficaram para uma versão 2.
- **Consequências:** o Android avisa "sem internet" e pode desviar o tráfego para os dados móveis: é preciso tocar em "Manter conexão" ou desligar os dados móveis. Para parecer app, a página tem manifest e ícone ("Adicionar à tela inicial").
- **Código:** `src/web.cpp`, `src/web/`, `platformio.ini`.

D-03 — `WebServer` do core + `WebSocketsServer` (links2004)

- **Contexto:** escolher a pilha de servidor.
- **Decisão:** `WebServer` (já vem no core Arduino-ESP32) para a página e `links2004/WebSockets` para o WebSocket.
- **Alternativas:** `ESPAsyncWebServer`, que tem vários forks e versões incompatíveis entre si.
- **Consequências:** os dois servidores são síncronos e rodam dentro do `loop()` (`web_atualizar`); um envio lento bloqueia o loop. Isso motivou a D-12.
- **Código:** `src/web.cpp`.

D-04 — Driver próprio do MPU-6050, sem a biblioteca `MPU6050_light`

- **Contexto:** o plano inicial usava a `MPU6050_light`. Com o sensor em pé nas costas, a fórmula de ângulo dela salta ~20° perto da vertical quando há inclinação lateral; o `calcOffsets()` dela supõe o sensor deitado; e o endereço 0x68 é fixo (a montagem BioSpine usa 0x69).

- **Decisão:** driver mínimo por registradores: procura em 0x68 e 0x69, configura ±250 °/s, ±2 g e DLPF de ~44 Hz, lê 14 bytes de uma vez, trata clones da família MPU-6500.
- **Alternativas:** outras bibliotecas também entregam ângulos de Euler e herdariam o problema da D-05.
- **Consequências:** cerca de 150 linhas a manter; os valores de registrador seguem o datasheet (rev. 4.2).
- **Código:** `src/mpu.cpp` — 03.

D-05 — Desvio = ângulo entre vetores gravidade; a calibração grava `g0`

- **Contexto:** ângulos de Euler (pitch/roll) têm singularidade justamente na posição em que o sensor fica no colete.
- **Decisão:** a medida de postura é o ângulo entre a gravidade atual e a gravidade gravada na calibração (`atan2(|a×b|, a·b)`). A calibração grava o vetor `g0`, e não `pitch0/roll0`. Frente/trás e lateral são calculados só para exibir.
- **Alternativas:** pitch/roll com troca de eixos conforme a montagem: frágil, depende de como o sensor foi preso.
- **Consequências:** funciona em qualquer orientação de montagem; o desvio não tem sinal (o alerta não sabe se foi para a frente ou para o lado, e não precisa saber).
- **Código:** `src/tilt.cpp` (`angleBetweenDeg`, `tiltComponentsDeg`), `src/settings.h` (`Calibracao::g0`).

D-06 — Filtro complementar sobre o vetor gravidade, com rejeição de aceleração

- **Contexto:** acelerômetro é ruidoso e sente o movimento; giroscópio deriva.
- **Decisão:** filtro complementar aplicado ao vetor (previsão `g × ω` pelo giroscópio + 2% do acelerômetro a cada 5 ms; τ ≈ 0,25 s). Quando |a| sai de [0,7; 1,3] g, o acelerômetro é ignorado naquele instante.
- **Alternativas:** Kalman ou Madgwick resolvem também, com mais parâmetros e mais custo; para inclinação lenta do tronco o complementar basta.
- **Consequências:** um único ajuste (`MPU_FILTRO_GIRO`); o sinal da convenção do giroscópio precisa ser conferido no hardware (`VALIDACAO.md`, V2.6).
- **Código:** `src/tilt.cpp` (`TiltFilter`).

D-07 — Offset do giroscópio no boot e recalculado sozinho quando parado

- **Contexto:** o colete pode ser ligado já vestido, com a pessoa se mexendo, e o boot não consegue medir o offset.
- **Decisão:** mede no boot (~1 s) só se a placa estiver parada; depois recalcula a cada ~1 s parado.
- **Consequências:** sem intervenção do usuário; o filtro complementar cobre o tempo sem offset.
- **Código:** `src/mpu.cpp` (`calibrarGiro`, `acumularOffset`).

D-08 — Detecção com limite, histerese e tempo mínimo (ok → atenção → alerta)

- **Contexto:** um limite simples liga e desliga o motor quando o desvio oscila perto dele, e vibra a cada movimento rápido.
- **Decisão:** entra em "ruim" acima do limite, sai só abaixo de limite − histerese; o motor só liga depois de `tempoMs` em "ruim". MPU e flex combinados com OU para entrar e E para sair.
- **Consequências:** três parâmetros ajustáveis; um estado intermediário (atenção) que o app mostra com contagem regressiva.
- **Código:** `src/posture.cpp` (`DetectorPostura`) — 04.

D-09 — Sem calibração, o colete não vibra

- **Contexto:** sem referência não existe "postura correta"; a postura de cada pessoa e a posição do sensor variam.
- **Decisão:** o detector força o estado ok enquanto não houver calibração válida. O app e a serial pedem a calibração.
- **Consequências:** primeiro uso exige apertar o botão; a calibração fica na NVS e vale nos próximos usos.
- **Código:** `DetectorPostura::atualizar`, `console_iniciar`.

D-10 — Calibração com espera, média e recusa se mexer; retorno pela vibração

- **Contexto:** o botão fica no colete; ao apertar, a pessoa mexe o tronco. Uma referência gravada em movimento faria o colete vibrar na postura correta.
- **Decisão:** 1 pulso ao começar, espera de 1,5 s, média de 1 s; recusa com pulso longo se o giroscópio passar de 30 °/s; 2 pulsos quando conclui. Calibração recusada mantém a anterior.
- **Consequências:** dá para calibrar sem olhar nenhuma tela.
- **Código:** `src/main.cpp` (`iniciarCalibracao`, `concluirCalibracao`), `src/posture.cpp` (`Calibrador`).

D-11 — Sensores analógicos opcionais e detectados sozinhos; flex 2 desligado por padrão; FSR só telemetria

- **Contexto:** a montagem pode vir sem flex, com flex quebrado ou sem resistor, com um ou dois flex, com ou sem FSR. O colete não pode depender do flex.
- **Decisão:** cada canal só entra na decisão se a leitura for plausível e estável por 2 s e se a calibração foi feita com ele ligado; no modo `auto` (padrão) sai sozinho após 1 s inválido (no modo `on` fica, mesmo solto). O flex 2 começa em `off` porque um GPIO35 solto copia a tensão do GPIO34 (capacitor do ADC) e pareceria um sensor de verdade. O FSR não alerta.
- **Consequências:** quem tem o segundo flex precisa de `set flex2 auto` e recalibrar.
- **Código:** `src/sensors.cpp`, `src/filters.h`, `src/settings.cpp` (`configPadrao`), `src/main.cpp` (`sensorAtivo`) — 05.

D-12 — Duas tarefas FreeRTOS: controle (prioridade 3) e loop (Wi-Fi e serial)

- **Contexto:** quando o celular sai do alcance, o envio TCP do WebSocket pode bloquear o `loop()` por segundos. Com tudo no loop, o colete pararia de detectar e vibrar nesse tempo.
- **Decisão:** a tarefa de controle (sensores, postura, motor, botão, calibração) roda a cada 1 ms com prioridade maior que o loop. As tarefas só conversam por pedidos, fila de eventos e cópias protegidas por trava. A tarefa de controle nunca chama `Serial` nem `web_*`.
- **Consequências:** regra rígida para quem mexer no código (usar `emitir()`); concorrência só validável no hardware.
- **Código:** `src/main.cpp` — 02.

D-13 — Agendamento por `millis()`, sem `delay()`

- **Contexto:** vários períodos diferentes (5, 10, 20, 100 ms) no mesmo laço.
- **Decisão:** cada etapa guarda o instante da última execução e roda quando passou o seu período. `delay()` só no boot (offset do giroscópio, reset do MPU), no `Mpu6050::iniciar` (~150 ms, também usado na reconexão do MPU, dentro da tarefa de controle) e no simulador.
- **Código:** `sensores_atualizar`, `passoControle`, `motor_atualizar`, `loop`.

D-14 — Parâmetros ajustáveis sem recompilar, validados no firmware e salvos na NVS

- **Contexto:** os limites certos só aparecem com o colete no corpo da pessoa, no dia do teste.
- **Decisão:** limite, histerese, tempo, limite do flex, intensidade e modos dos sensores mudam pela serial e pelo app. O firmware valida (`validarParams`) qualquer valor recebido e grava na NVS.
- **Consequências:** a validação fica num lugar só; a página pode limitar os controles, mas quem garante é o firmware.
- **Código:** `src/posture.cpp`, `src/settings.cpp`, `src/console.cpp`, `src/protocolo.cpp` — 06.

D-15 — Motor por PWM a 20 kHz, alerta em pulsos de 300/700 ms, LED junto

- **Contexto:** vibração contínua incomoda e o corpo se acostuma; PWM audível faz o motor chiar.
- **Decisão:** PWM fora da faixa audível, alerta em pulsos, LED da placa como espelho do motor para diagnóstico.
- **Código:** `src/motor.cpp`.

D-16 — Wi-Fi com potência reduzida (11 dBm), sem economia de energia, heartbeat no WebSocket

- **Contexto:** pico de corrente do Wi-Fi somado ao motor pode derrubar a tensão e reiniciar o ESP32; celulares somem da rede sem avisar.
- **Decisão:** `WiFi.setTxPower(WIFI_POWER_11dBm)` (alcance de sobra para uma sala), `WiFi.setSleep(false)` (latência estável), heartbeat de 5 s que derruba clientes que sumiram.
- **Código:** `src/web.cpp` (`web_iniciar`).

D-17 — I2C a 100 kHz, dois endereços e reconexão automática do MPU

- **Contexto:** fios longos no colete, mau contato, duas montagens com AD0 diferente.
- **Decisão:** 100 kHz (mais tolerante que 400 kHz), timeout de 20 ms, endereços 0x68 e 0x69, 10 falhas seguidas = sensor perdido, nova tentativa a cada 2 s, avisos na serial e no app.
- **Código:** `src/sensors.cpp`, `src/mpu.cpp`.

D-18 — Um firmware para as duas montagens (GUIA e BioSpine V3)

- **Contexto:** existem duas pinagens em uso (a do `montagem.pdf` e a do README do BioSpine V3), e nenhum pino conflita.
- **Decisão:** atender as duas ao mesmo tempo, com detecção automática do que está ligado, e aceitar os comandos serial do BioSpine V3 (`START`, `STOP`, `STATUS`, `CALIBRATE`) com as respostas JSON no formato dele.
- **Código:** `src/config.h`, `src/console.cpp` — 07.

D-19 — Lógica pura separada do hardware; testes no PC e simulador do firmware inteiro

- **Contexto:** o hardware só ficou disponível no último dia.
- **Decisão:** `tilt`, `posture` e `filters` não dependem do Arduino e têm 21 testes nativos. Um simulador compila o firmware real no PC, trocando só o hardware (MPU falso, ADC, botão, NVS em memória, Wi-Fi desligado).
- **Consequências:** quase todos os fluxos foram exercitados antes da placa; continuam sem cobertura a concorrência, o ADC real e o Wi-Fi real.
- **Código:** `test/`, `sim/`, `tools/sim.sh` — 09.

D-20 — Protocolo JSON num módulo separado do Wi-Fi

- **Contexto:** o simulador não tem Wi-Fi, mas o JSON do app precisava ser testado.
- **Decisão:** `protocolo.cpp` monta e interpreta as mensagens (ArduinoJson) sem conhecer o Wi-Fi; `web.cpp` só transporta. O simulador e o `tools/sim_web.py` usam o mesmo `protocolo.cpp` com a página real.
- **Código:** `src/protocolo.cpp`, `src/web.cpp`, `sim/sim_main.cpp`.